

IPA ASSIGNMENT 13

Name – Nishant Nahar

Roll No – 241551078

1. Ideal Low Pass Filter

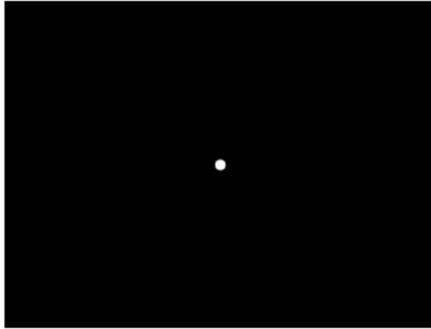
```
import cv2
import numpy as np
import matplotlib.pyplot as plt
img = cv2.imread("img_3.jpeg", cv2.IMREAD_GRAYSCALE)
if img is None:
    raise FileNotFoundError("Image not found!")
F = np.fft.fft2(img)
F_shift = np.fft.fftshift(F)
rows, cols = img.shape
crow, ccol = rows // 2, cols // 2
radii = [10, 25, 50, 100]
plt.figure(figsize=(12, 12))
for i, radius in enumerate(radii):
    mask = np.zeros((rows, cols), np.uint8)
    cv2.circle(mask, (ccol, crow), radius, 1, -1)
    F_filtered = F_shift * mask
    F_ishift = np.fft.ifftshift(F_filtered)
    img_filtered = np.fft.ifft2(F_ishift)
    img_filtered = np.abs(img_filtered)
    spectrum = np.log(1 + np.abs(F_filtered))
    plt.subplot(4, 3, i * 3 + 1)
    plt.imshow(img_filtered, cmap="gray")
    plt.title(f"ILPF - Radius {radius}")
    plt.axis("off")
    plt.subplot(4, 3, i * 3 + 2)
    plt.imshow(mask, cmap="gray")
    plt.title(f"Filter Mask - R={radius}")
    plt.axis("off")
    plt.subplot(4, 3, i * 3 + 3)
    plt.imshow(spectrum, cmap="gray")
    plt.title(f"FT - Radius {radius}")
    plt.axis("off")
plt.tight_layout()
plt.show()
```

OUTPUT

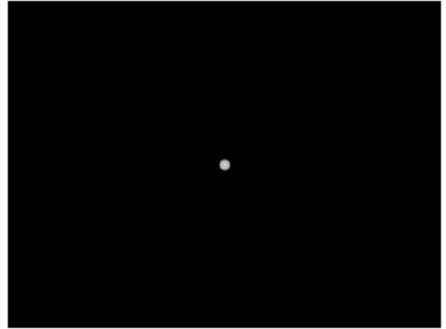
ILPF - Radius 10



Filter Mask - R=10



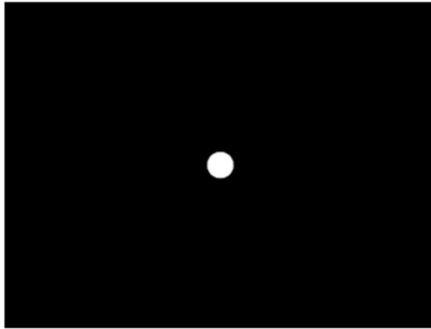
FT - Radius 10



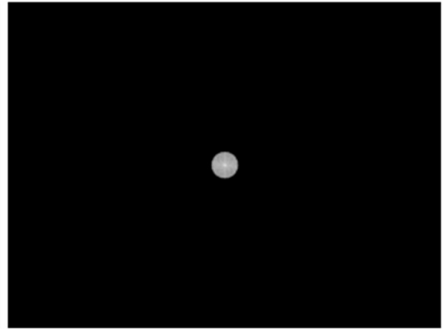
ILPF - Radius 25



Filter Mask - R=25



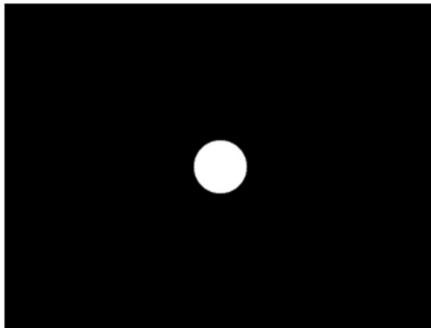
FT - Radius 25



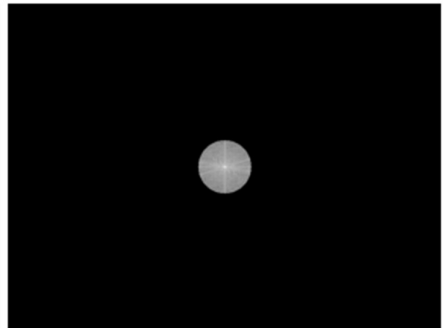
ILPF - Radius 50



Filter Mask - R=50



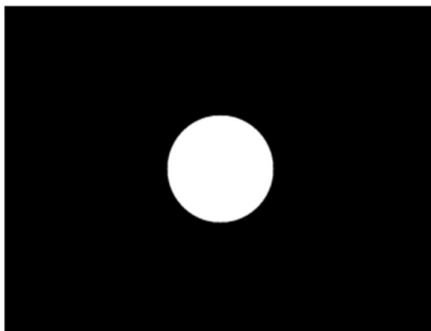
FT - Radius 50



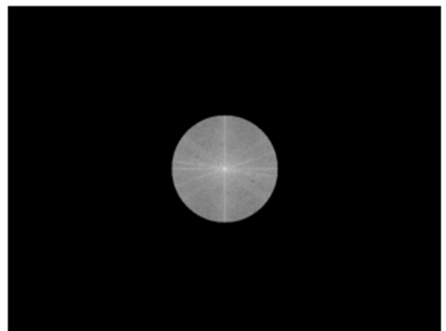
ILPF - Radius 100



Filter Mask - R=100



FT - Radius 100



2. Blurring at different radius - 10,25,50,100

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
img = cv2.imread("img_3.jpeg", cv2.IMREAD_GRAYSCALE)
if img is None:
    raise FileNotFoundError("Image not found!")
blur_levels = [25, 50, 75, 100]
plt.figure(figsize=(12, 12))
for i, level in enumerate(blur_levels):
    kernel = level
    if kernel % 2 == 0:
        kernel += 1
    blurred = cv2.GaussianBlur(img, (kernel, kernel), 0)
    f = np.fft.fft2(blurred)
    f_shift = np.fft.fftshift(f)
    magnitude = np.abs(f_shift)
    ft = np.log(1 + magnitude)
    plt.subplot(4, 2, 2 * i + 1)
    plt.imshow(blurred, cmap="gray")
    plt.title(f"Blur Level = {level}")
    plt.axis("off")
    plt.subplot(4, 2, 2 * i + 2)
    plt.imshow(ft, cmap="gray")
    plt.title(f"FT - Blur Level = {level}")
    plt.axis("off")
plt.tight_layout()
plt.show()
```

OUTPUT

Blur Level = 25



Blur Level = 50



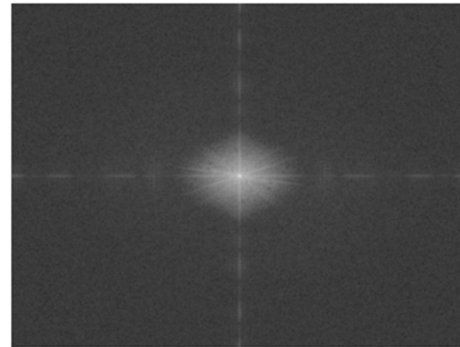
Blur Level = 75



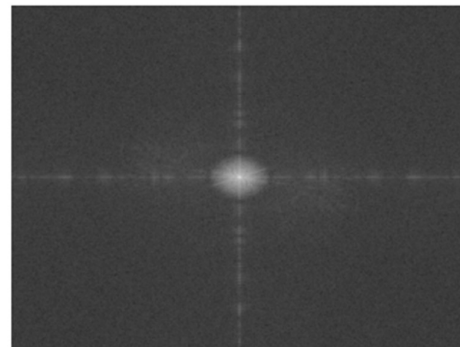
Blur Level = 100



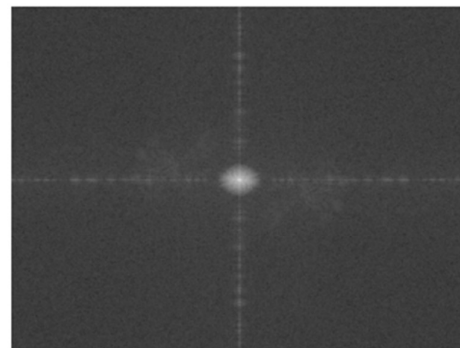
FT - Blur Level = 25



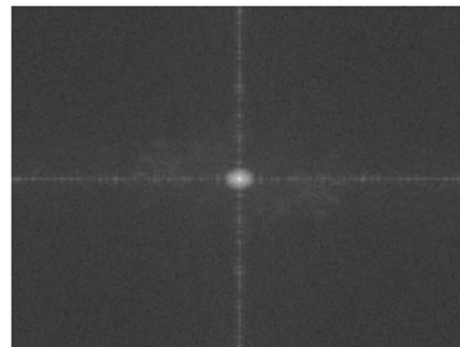
FT - Blur Level = 50



FT - Blur Level = 75



FT - Blur Level = 100



3. 2D Fourier Transform - 3 Images

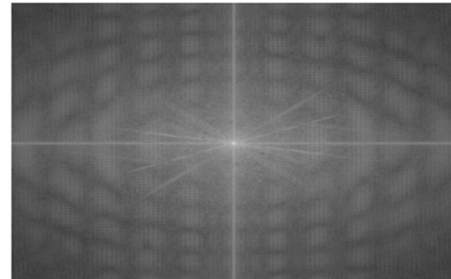
```
import cv2
import numpy as np
import matplotlib.pyplot as plt
image_files = ["img_1.jpeg", "img_2.jpeg", "img_3.jpeg"]
plt.figure(figsize=(12, 10))
for i, file in enumerate(image_files):
    img = cv2.imread(file, cv2.IMREAD_GRAYSCALE)
    if img is None:
        print(f"Could not read {file}")
        continue
    f = np.fft.fft2(img)
    f_shift = np.fft.fftshift(f)
    magnitude = np.abs(f_shift)
    magnitude_spectrum = np.log(1 + magnitude)
    plt.subplot(4, 2, 2 * i + 1)
    plt.imshow(img, cmap="gray")
    plt.title(f"Original Image {i+1}")
    plt.axis("off")
    plt.subplot(4, 2, 2 * i + 2)
    plt.imshow(magnitude_spectrum, cmap="gray")
    plt.title(f"2D Fourier Transform {i+1}")
    plt.axis("off")
plt.tight_layout()
plt.show()
```

OUTPUT

Original Image 1



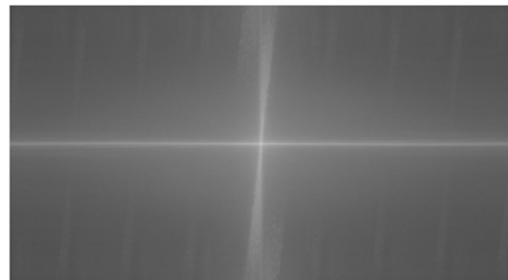
2D Fourier Transform 1



Original Image 2



2D Fourier Transform 2



Original Image 3



2D Fourier Transform 3



4. Butterworth Low Pass Filter

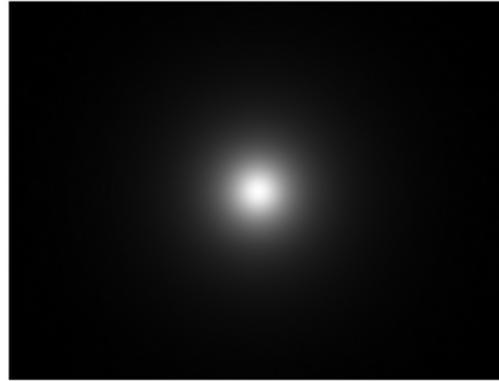
```
import cv2
import numpy as np
import matplotlib.pyplot as plt
img = cv2.imread("img_3.jpeg", cv2.IMREAD_GRAYSCALE)
if img is None:
    raise FileNotFoundError("Image not found!")
F = np.fft.fft2(img)
F_shift = np.fft.fftshift(F)
rows, cols = img.shape
crow, ccol = rows // 2, cols // 2
u = np.arange(cols)
v = np.arange(rows)
U, V = np.meshgrid(u, v)
D = np.sqrt((U - ccol) ** 2 + (V - crow) ** 2)
D0 = 50
orders = [1, 2, 4, 8]
plt.figure(figsize=(12, 10))
for i, n in enumerate(orders):
    H = 1 / (1 + (D / D0) ** (2 * n))
    F_filtered = F_shift * H
    F_ishift = np.fft.ifftshift(F_filtered)
    img_filtered = np.fft.ifft2(F_ishift)
    img_filtered = np.abs(img_filtered)
    spectrum = np.log(1 + np.abs(F_filtered))
    plt.subplot(4, 2, 2 * i + 1)
    plt.imshow(img_filtered, cmap="gray")
    plt.title(f"Butterworth LPF, n = {n}")
    plt.axis("off")
    plt.subplot(4, 2, 2 * i + 2)
    plt.imshow(H, cmap="gray")
    plt.title(f"Filter, n = {n}")
    plt.axis("off")
plt.tight_layout()
plt.show()
```

OUTPUT

Butterworth LPF, $n = 1$



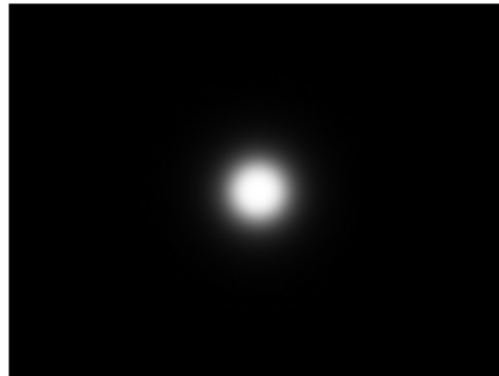
Filter, $n = 1$



Butterworth LPF, $n = 2$



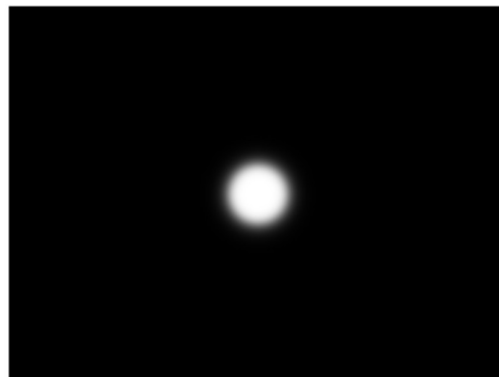
Filter, $n = 2$



Butterworth LPF, $n = 4$



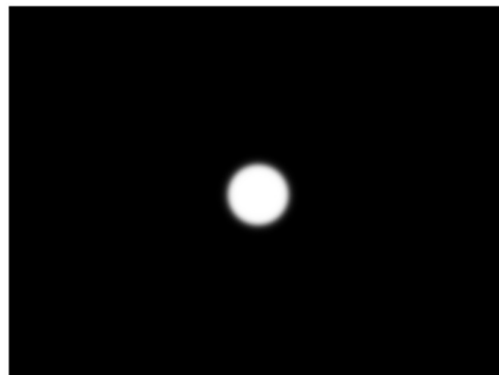
Filter, $n = 4$



Butterworth LPF, $n = 8$



Filter, $n = 8$



5. Gaussian Low Pass Filter

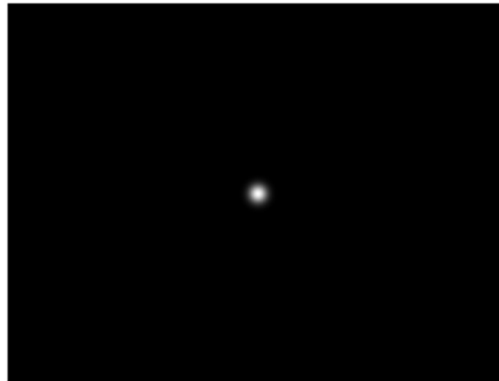
```
import cv2
import numpy as np
import matplotlib.pyplot as plt
img = cv2.imread("img_3.jpeg", cv2.IMREAD_GRAYSCALE)
if img is None:
    raise FileNotFoundError("Image not found!")
F = np.fft.fft2(img)
F_shift = np.fft.fftshift(F)
rows, cols = img.shape
crow, ccol = rows // 2, cols // 2
u = np.arange(cols)
v = np.arange(rows)
U, V = np.meshgrid(u, v)
D = np.sqrt((U - ccol) ** 2 + (V - crow) ** 2)
sigma_values = [10, 25, 50, 100]
plt.figure(figsize=(12, 10))
for i, sigma in enumerate(sigma_values):
    H = np.exp(-(D**2) / (2 * sigma**2))
    F_filtered = F_shift * H
    F_ishift = np.fft.ifftshift(F_filtered)
    img_filtered = np.fft.ifft2(F_ishift)
    img_filtered = np.abs(img_filtered)
    spectrum = np.log(1 + np.abs(F_filtered))
    plt.subplot(4, 2, 2 * i + 1)
    plt.imshow(img_filtered, cmap="gray")
    plt.title(f"Gaussian LPF,  $\sigma = \{sigma\}$ ")
    plt.axis("off")
    plt.subplot(4, 2, 2 * i + 2)
    plt.imshow(H, cmap="gray")
    plt.title(f"Gaussian Filter,  $\sigma = \{sigma\}$ ")
    plt.axis("off")
plt.tight_layout()
plt.show()
```

OUTPUT

Gaussian LPF, $\sigma = 10$



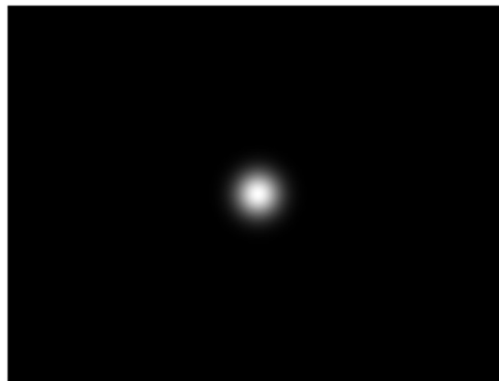
Gaussian Filter, $\sigma = 10$



Gaussian LPF, $\sigma = 25$



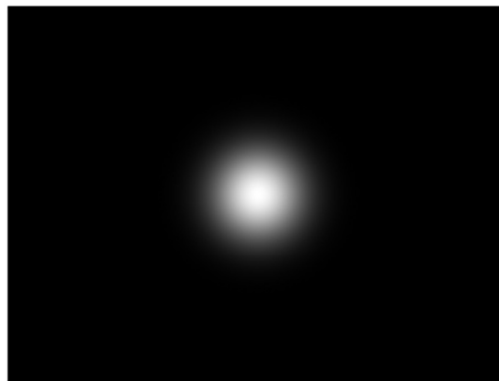
Gaussian Filter, $\sigma = 25$



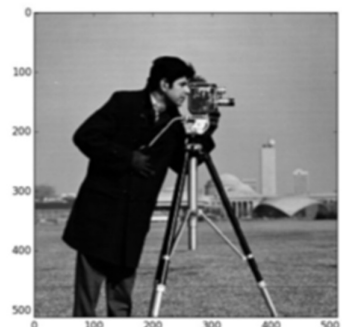
Gaussian LPF, $\sigma = 50$



Gaussian Filter, $\sigma = 50$



Gaussian LPF, $\sigma = 100$



Gaussian Filter, $\sigma = 100$



6. High Pass Filter All the 3 types

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
img = cv2.imread("img_3.jpeg", cv2.IMREAD_GRAYSCALE)
if img is None:
    raise FileNotFoundError("Image not found!")
F = np.fft.fft2(img)
F_shift = np.fft.fftshift(F)
rows, cols = img.shape
crow, ccol = rows // 2, cols // 2
u = np.arange(cols)
v = np.arange(rows)
U, V = np.meshgrid(u, v)
D = np.sqrt((U - ccol) ** 2 + (V - crow) ** 2) D0 = 50
n = 2
D[D == 0] = 1e-5
H_ideal = np.zeros((rows, cols))
H_ideal[D > D0] = 1
F_ideal = F_shift * H_ideal
img_ideal = np.fft.ifft2(np.fft.ifftshift(F_ideal))
img_ideal = np.abs(img_ideal)
H_butterworth = 1 / (1 + (D0 / D) ** (2 * n))
F_butterworth = F_shift * H_butterworth
img_butterworth = np.fft.ifft2(np.fft.ifftshift(F_butterworth))
img_butterworth = np.abs(img_butterworth)
H_gaussian = 1 - np.exp(-(D**2) / (2 * D0**2))
F_gaussian = F_shift * H_gaussian
img_gaussian = np.fft.ifft2(np.fft.ifftshift(F_gaussian))
img_gaussian = np.abs(img_gaussian)
spectrum_ideal = np.log(1 + np.abs(F_ideal))
spectrum_butterworth = np.log(1 + np.abs(F_butterworth))
spectrum_gaussian = np.log(1 + np.abs(F_gaussian))
plt.figure(figsize=(15, 12))
plt.subplot(4, 4, 1)
plt.imshow(img, cmap="gray")
plt.title("Original Image")
plt.axis("off")
plt.subplot(4, 4, 2)
plt.imshow(np.log(1 + np.abs(F_shift)), cmap="gray")
plt.title("Original FT")
plt.axis("off")
plt.subplot(4, 4, 3)
plt.imshow(H_ideal, cmap="gray")
plt.title("Ideal HPF")
plt.axis("off")
plt.subplot(4, 4, 4)
plt.imshow(img_ideal, cmap="gray")
plt.title("Ideal HPF Result")
plt.axis("off")
plt.subplot(4, 4, 5)
plt.imshow(H_butterworth, cmap="gray")
plt.title(f"Butterworth HPF (n={n})")
plt.axis("off")
plt.subplot(4, 4, 6)
```

```

plt.imshow(img_butterworth, cmap="gray")
plt.title("Butterworth Result")
plt.axis("off")
plt.subplot(4, 4, 7)
plt.imshow(spectrum_butterworth, cmap="gray")
plt.title("Butterworth FT")
plt.axis("off")
plt.subplot(4, 4, 9)
plt.imshow(H_gaussian, cmap="gray")
plt.title("Gaussian HPF")
plt.axis("off")
plt.subplot(4, 4, 10)
plt.imshow(img_gaussian, cmap="gray")
plt.title("Gaussian Result")
plt.axis("off")
plt.subplot(4, 4, 11)
plt.imshow(spectrum_gaussian, cmap="gray")
plt.title("Gaussian FT")
plt.axis("off")
plt.subplot(4, 4, 12)
plt.imshow(spectrum_ideal, cmap="gray")
plt.title("Ideal HPF FT")
plt.axis("off")
plt.tight_layout()
plt.show()

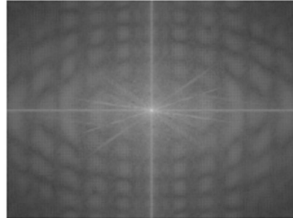
```

OUTPUT

Original Image



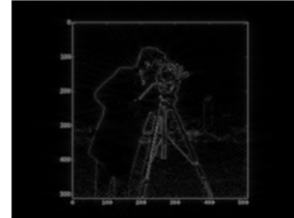
Original FT



Ideal HPF



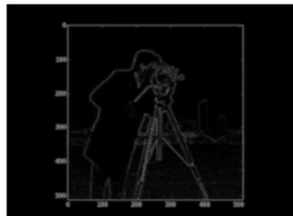
Ideal HPF Result



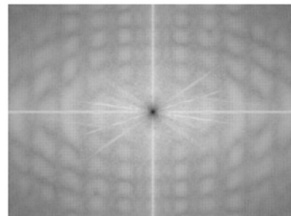
Butterworth HPF (n=2)



Butterworth Result



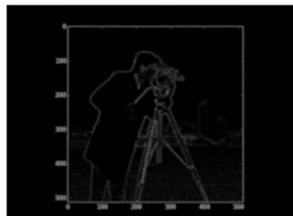
Butterworth FT



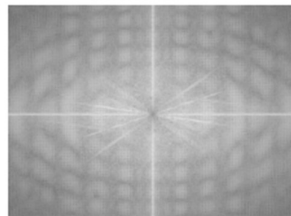
Gaussian HPF



Gaussian Result



Gaussian FT



Ideal HPF FT

